P25 GPS Tracker — Software Security Summary

A plain-language security review for IT, in the spirit of a cyber-safety audit.

Tool: **P25 GPS Tracker v0.4** · Author: Jim Gaddy (built with AI assistance) · Source: public, open-source · 2026-11-11

> **In one line:** a small, open-source Python program that **reads** GPS from our own BK P25 radios over a USB cable and draws each unit on a local map (ArcGIS Earth). It uses **only the Python standard library** (no third-party packages, nothing downloaded from the internet), makes **no internet or cloud connection**, **opens no listening ports**, needs **no administrator rights**, and writes only to its own folder. It is a firefighter-safety tool intended to run on a single command-post PC.

1. Why this document exists

Our group-policy workstations do not have Python installed and restrict scripts. Before IT is asked to allow this tool, this document states **exactly** what it is, what it does, what it touches on the network and the disk, and what it does not do — so IT can make its own determination and choose a deployment path that fits policy. Every claim below is verifiable against the source code (file hashes in the appendix).

2. What Python is (the runtime)

Python is a mainstream, open-source programming language from the non-profit **Python Software Foundation** (python.org). It is one of the most widely used languages in government, enterprise, and science. A Python program is a plain-text file (`.py`) that the Python interpreter (`python.exe`) reads and runs **with the signed-in user's own privileges — not elevated.** Python can be installed per-user without administrator rights, or deployed and managed by IT.

On "no scripts": Group Policy script restrictions usually target Windows Script Host (VBScript/JScript), batch files, and PowerShell's execution policy. Python is a separate application runtime; whether `python.exe` is permitted is governed by application controls such as AppLocker or Software Restriction Policy, which **IT controls.** Section 9 lists ways to run this tool that fit a restrictive environment, including as a single compiled executable with no Python runtime at all.

3. What the software does

1. Opens one outbound connection to our base radio (which appears as a USB network adapter) and sends a 5-byte "start sending GPS" request.
2. Reads the GPS the radio relays — standard NMEA sentences carrying position and the sending radio's unit ID.
3. Draws one marker per unit on **ArcGIS Earth** (Esri's free desktop globe, running on the same PC) through its local API at `http://localhost:8000`.

4. Writes a local log of each fix (time, unit, position) and a small alias/settings file.

It is strictly read-and-display: it never sends anything to the radios except the one request to begin the GPS stream, and it never reprograms a radio.

4. Network behavior

This is usually IT's first question. The tool is a **client only** — it makes two outbound connections and **listens on nothing.**

Connects to	What that is	Direction	Leaves the PC?
`10.250.46.6 : 65535`	The base radio itself, reached over the USB cable (RNDIS). This address exists only on the USB link between the PC and the radio — it is not on the agency network and is not routable to the internet.	Outbound	**No — stays on the USB cable**
`127.0.0.1 : 8000`	ArcGIS Earth, running on the same PC (localhost).	Outbound	**No — never leaves the machine**
Any internet / cloud host	— none —	**NONE**	**—**
Any inbound / listening port	— none — the tool runs no server and accepts no connections	**NONE**	**—**

The tool deliberately bypasses the system proxy **for localhost only**, so its map calls go straight to ArcGIS Earth on the same machine and are never routed outward. There is no telemetry, no update check, and no analytics.

5. Data handling

- **Reads:** radio unit IDs (numbers) and GPS position (lat, lon, altitude, satellite count) from our own radios.
- **Writes:** only to the tool's own folder — a friendly-name file (`aliases.json`), a settings file (`settings.json`), and a dated CSV + text log of fixes.
- **Stores no credentials** and no personal data beyond radio IDs and positions. Nothing is transmitted off the PC; the log is a local file the agency owns and controls.

6. Privileges and system changes

Runs as a **standard user**. It does **not**: request or require administrator rights (no UAC prompt); install a Windows service; create a scheduled task or auto-start entry; change the registry; change any system or security setting; or install a device driver (the radio uses Windows' **built-in** "USB Ethernet/RNDIS Gadget" driver). It reads and writes only inside its own folder and does not touch other applications' data.

7. What the software does NOT do

×	Connect to the internet or any cloud service	×	Open any listening/inbound network port
×	Phone home, send telemetry, or auto-update	×	Require or request administrator elevation
×	Install services, drivers, or startup entries	×	Modify system, registry, or security settings
×	Read other applications' files or user data	×	Encrypt, delete, or move files outside its folder
×	Download or run any code from the internet	×	Reprogram or write settings to the radios

8. Dependencies and supply chain

The P25 GPS Tracker uses **only the Python standard library** — the modules that ship inside Python itself (`socket, urllib, json, csv, re, tkinter, threading, queue, datetime, pathlib`). It installs **no third-party packages** and downloads **nothing** from PyPI or any other repository. This removes the usual open-source supply-chain concern: the only software to install is the official **CPython** interpreter from python.org (Python Software Foundation) or from IT's own managed source.

> **Full disclosure:** an optional companion build that also reads a Raveon radio on a serial/COM port (the "PRAVE / ME / P25" variant) adds one widely used package, `pyserial`. That package is **not** part of the standalone P25 GPS Tracker described here. ArcGIS Earth is Esri's product and is reviewed/approved separately; this tool only uses its on-PC API.

9. Ways to run it within managed policies

Three paths, in order of least effort for IT. Any one works — IT's call:

1. **One command-post PC.** Run it on the single dispatch/command machine — ideally a PC not under the most restrictive user policy. Fewest moving parts; nothing changes on fleet workstations.
2. **IT-managed Python.** IT installs CPython centrally and allows (AppLocker / SRP) `python.exe` plus this tool's folder — standard application allow-listing.
3. **Single compiled executable.** If a "no interpreters / no scripts" rule is the blocker, the tool can be delivered as one Windows `.exe` — no Python runtime and no script file to run.

> **Honest note on option 3:** a freshly built `.exe` is unsigned, so Windows SmartScreen and AppLocker will treat it like any newly built application until IT allow-lists or signs it. It removes the Python question but does not remove IT's normal application-control step.

10. Auditability

The complete source is public and small — **two files, 712 lines total**, plain text, readable in well under a day. The SHA-256 hashes below let IT confirm the files are exactly the reviewed version and have not been altered.

File	Lines	SHA-256
`p25_core.py`	385	`0be347dcff78d5423cf0421c0f36afb9a5d4ecfcbb12dbab9cda3770b4d22f1a`
`p25_tracker_app.py`	327	`1250646fb89ee477eb9e3c03c8a3215beea3bcea2d9bb2df5fde30743b0d931a`

Verify on Windows with: `Get-FileHash .\p25_core.py -Algorithm SHA256`

11. Summary and request

The P25 GPS Tracker is a **read-only, local-only, open-source** firefighter-safety tool with **no third-party dependencies**, **no internet access**, **no listening ports**, and **no need for administrator rights**, intended for a single command-post PC. We ask IT to review the facts above and choose a deployment path from Section 9 that fits agency policy. All source, hashes, and this document are available for a full review.

Software security summary for IT review — P25 GPS Tracker v0.4, built 2026-11-11. Facts in this document are verifiable against the published source code. This summary describes the software's behavior; the deployment decision rests with IT.